Laporan Pemrograman Mobile 7



Nama : **David Yehezkiel**
NIM : **20240801013**
Prodi : **Teknik Informatika**

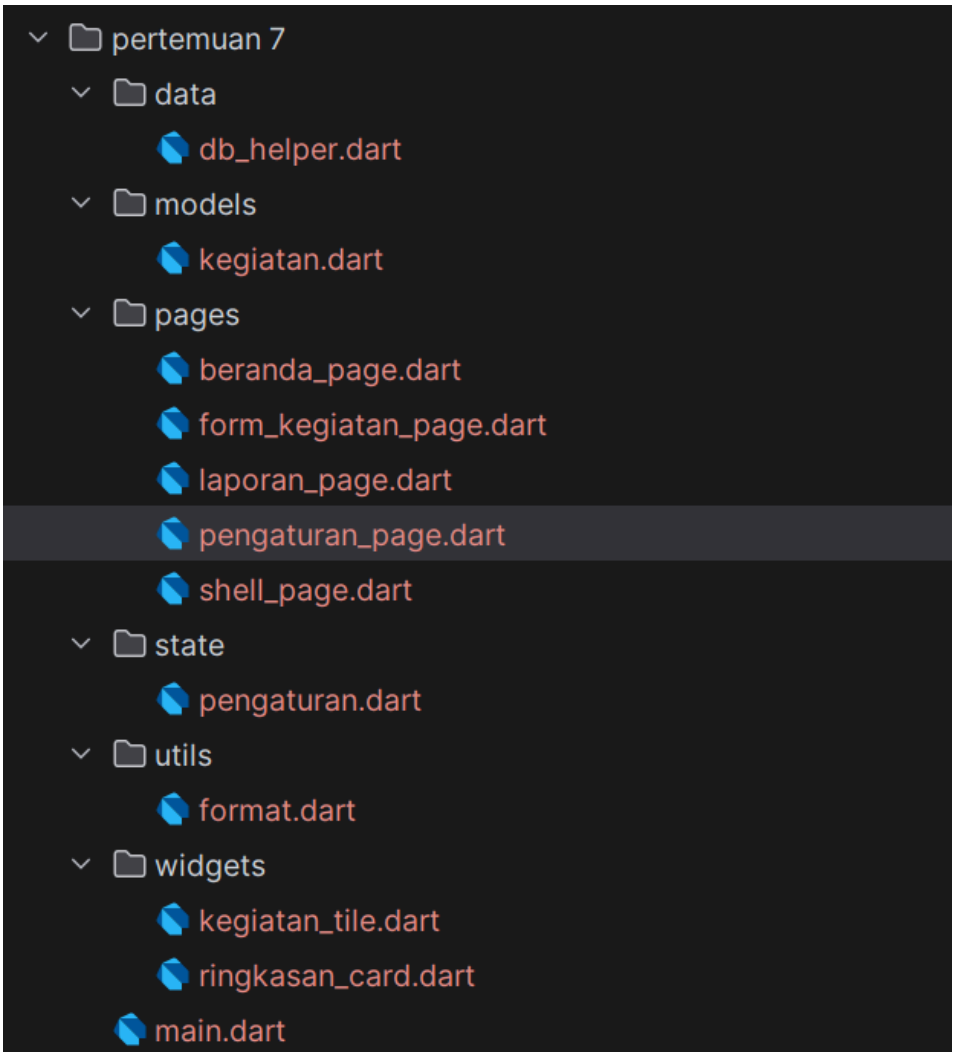
Universitas Esa Unggul
Kampus Citra Raya
2026

Pertanyaan Refleksi :

Pasang Package :

```
PS C:\Users\advan\StudioProjects\untitled> flutter pub add sqflite path shared_preferences
"sqflite" is already in "dependencies". Will try to update the constraint.
"path" is already in "dependencies". Will try to update the constraint.
"shared_preferences" is already in "dependencies". Will try to update the constraint.
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  shared_preferences_android 2.4.28 (2.5.4 available)
  shared_preferences_foundation 2.5.7 (2.6.1 available)
> sqflite 2.4.5 (was 2.4.4+1)
> sqflite_android 2.4.5 (was 2.4.4)
  sqflite_common 2.5.13+1 (2.5.14 available)
  sqflite_darwin 2.4.4 (2.4.4+1 available)
> sqflite_platform_interface 2.4.3 (was 2.4.2)
  synchronized 3.4.2 (3.5.0 available)
  test_api 0.7.12 (0.7.15 available)
  vector_math 2.4.0 (2.4.4 available)
  vm_service 15.3.0 (15.4.0 available)
  web 1.1.1 (1.1.2 available)
Changed 3 dependencies!
12 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
```

Struktur file :



main.dart :

```dart
import 'package:flutter/material.dart';
import 'state/pengaturan.dart';
import 'pages/shell_page.dart';
import 'pages/form_kegiatan_page.dart';

void main() {
 runApp(const TabungKuApp());
}

class TabungKuApp extends StatefulWidget {
 const TabungKuApp({super.key});

 @override
 State<TabungKuApp> createState() => _TabungKuAppState();
}

class _TabungKuAppState extends State<TabungKuApp> {
 final Pengaturan pengaturan = Pengaturan();

 @override
 void initState() {
   super.initState();
   pengaturan.muat();
 }

 @override
 void dispose() {
   pengaturan.dispose();
   super.dispose();
 }

 @override
 Widget build(BuildContext context) {
   return AnimatedBuilder(
     animation: pengaturan,
     builder: (context, _) {
       return MaterialApp(
         debugShowCheckedModeBanner: false,
         title: 'TabungKu',
         theme: ThemeData(
           useMaterial3: true,
           colorSchemeSeed: pengaturan.warna,
           brightness: Brightness.light,
         ),
         darkTheme: ThemeData(
           useMaterial3: true,
           colorSchemeSeed: pengaturan.warna,
           brightness: Brightness.dark,
         ),
         themeMode:
         pengaturan.gelap ? ThemeMode.dark : ThemeMode.light,
         initialRoute: '/',
```

```
        routes: {
          '/': (context) => ShellPage(
            pengaturan: pengaturan,
          ),
          '/form': (context) => FormKegiatanPage(
            pengaturan: pengaturan,
          ),
        },
      );
    },
  );
 }
}
```

models/kegiatan.dart :

```
enum Tipe {
 tanpa,
 pemasukan,
 pengeluaran,
}

class Kegiatan {
 final int? id;
 final String judul;
 final String tanggal;
 final bool selesai;
 final Tipe tipe;
 final int nominal;
 final String kategori;

 const Kegiatan({
   this.id,
   required this.judul,
   required this.tanggal,
   required this.selesai,
   required this.tipe,
   required this.nominal,
   required this.kategori,
 });

 Map<String, Object?> toMap() {
   return {
     'id': id,
     'judul': judul,
     'tanggal': tanggal,
     'selesai': selesai ? 1 : 0,
     'tipe': tipe.name,
     'nominal': nominal,
     'kategori': kategori,
   };
 }

 factory Kegiatan.fromMap(Map<String, Object?> map) {
```

```
    return Kegiatan(
      id: map['id'] as int,
      judul: map['judul'] as String,
      tanggal: map['tanggal'] as String,
      selesai: (map['selesai'] as int) == 1,
      tipe: Tipe.values.byName(map['tipe'] as String),
      nominal: map['nominal'] as int,
      kategori: map['kategori'] as String,
    );
  }
}

class Ringkasan {
  final int jumlahKegiatan;
  final int selesai;
  final int pemasukan;
  final int pengeluaran;

  const Ringkasan({
    required this.jumlahKegiatan,
    required this.selesai,
    required this.pemasukan,
    required this.pengeluaran,
  });

  int get tabungan => pemasukan - pengeluaran;
}
```

data/db_helper.dart :

```
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';
import '../models/kegiatan.dart';

final ValueNotifier<int> versiData = ValueNotifier<int>(0);

class DbHelper {
  static Database? _db;

  static Future<Database> get database async {
    if (_db != null) {
      return _db!;
    }

    final path = p.join(
      await getDatabasesPath(),
      'tabungku.db',
    );

    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
```

```
        CREATE TABLE kegiatan(
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          judul TEXT NOT NULL,
          tanggal TEXT NOT NULL,
          selesai INTEGER NOT NULL,
          tipe TEXT NOT NULL,
          nominal INTEGER NOT NULL,
          kategori TEXT NOT NULL
        )
      ''');
    },
  );

  return _db!;
}

static Future<int> tambah(Kegiatan kegiatan) async {
  final db = await database;

  final id = await db.insert(
    'kegiatan',
    kegiatan.toMap(),
  );

  versiData.value++;
  return id;
}

static Future<int> ubah(Kegiatan kegiatan) async {
  final db = await database;

  final hasil = await db.update(
    'kegiatan',
    kegiatan.toMap(),
    where: 'id = ?',
    whereArgs: [kegiatan.id],
  );

  versiData.value++;
  return hasil;
}

static Future<int> hapus(int id) async {
  final db = await database;

  final hasil = await db.delete(
    'kegiatan',
    where: 'id = ?',
    whereArgs: [id],
  );

  versiData.value++;
  return hasil;
```

```
}

static Future<int> ubahSelesai(
    int id,
    bool selesai,
    ) async {
  final db = await database;

  final hasil = await db.update(
    'kegiatan',
    {'selesai': selesai ? 1 : 0},
    where: 'id = ?',
    whereArgs: [id],
  );

  versiData.value++;
  return hasil;
}

static Future<List<Kegiatan>> pada(
    String tanggal,
    ) async {
  final db = await database;

  final rows = await db.query(
    'kegiatan',
    where: 'tanggal = ?',
    whereArgs: [tanggal],
    orderBy: 'id DESC',
  );

  return rows.map(Kegiatan.fromMap).toList();
}

static Future<Ringkasan> ringkasan(
    String tanggal,
    ) async {
  final db = await database;

  final hasil = await db.rawQuery('''
    SELECT
      COUNT(*) AS jumlah,
      COALESCE(SUM(selesai), 0) AS selesai,
      COALESCE(
        SUM(
          CASE
            WHEN tipe = 'pemasukan'
            THEN nominal
            ELSE 0
          END
        ),
        0
      ) AS masuk,
```

```
      COALESCE(
        SUM(
          CASE
            WHEN tipe = 'pengeluaran'
            THEN nominal
            ELSE 0
          END
        ),
        0
      ) AS keluar
    FROM kegiatan
    WHERE tanggal = ?
  ''', [tanggal]);

  final baris = hasil.first;

  return Ringkasan(
    jumlahKegiatan: (baris['jumlah'] as int?) ?? 0,
    selesai: (baris['selesai'] as int?) ?? 0,
    pemasukan: (baris['masuk'] as int?) ?? 0,
    pengeluaran: (baris['keluar'] as int?) ?? 0,
  );
}

static Future<Map<String, int>> pengeluaranPerKategori(
    String tanggal,
    ) async {
  final db = await database;

  final hasil = await db.rawQuery('''
    SELECT
      kategori,
      COALESCE(SUM(nominal), 0) AS total
    FROM kegiatan
    WHERE tanggal = ?
      AND tipe = 'pengeluaran'
    GROUP BY kategori
    ORDER BY total DESC
  ''', [tanggal]);

  return {
    for (final row in hasil)
      row['kategori'] as String:
      (row['total'] as int?) ?? 0,
  };
}

static Future<List<Map<String, Object?>>> tujuhHariTerakhir(
    String tanggal,
    ) async {
  final db = await database;

  return db.rawQuery('''
```

```
    SELECT
      tanggal,
      COUNT(*) AS jumlah,
      COALESCE(
        SUM(
          CASE
            WHEN tipe = 'pemasukan'
            THEN nominal
            ELSE 0
          END
        ),
        0
      ) AS pemasukan,
      COALESCE(
        SUM(
          CASE
            WHEN tipe = 'pengeluaran'
            THEN nominal
            ELSE 0
          END
        ),
        0
      ) AS pengeluaran
    FROM kegiatan
    WHERE tanggal BETWEEN date(?, '-6 day') AND ?
    GROUP BY tanggal
    ORDER BY tanggal DESC
  ''', [tanggal, tanggal]);
}

static Future<List<Kegiatan>> cari(
    String kata,
    ) async {
  final db = await database;

  final rows = await db.query(
    'kegiatan',
    where: '''
      judul LIKE ?
      OR tipe LIKE ?
      OR kategori LIKE ?
    ''',
    whereArgs: [
      '%$kata%',
      '%$kata%',
      '%$kata%',
    ],
    orderBy: 'tanggal DESC, id DESC',
  );

  return rows.map(Kegiatan.fromMap).toList();
}
}
```

data/db_helper.dart :

```dart
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';
import '../models/kegiatan.dart';
import 'package:flutter/foundation.dart';

final ValueNotifier<int> versiData = ValueNotifier<int>(0);

class DbHelper {
 static Database? _db;

 static Future<Database> get database async {
   if (_db != null) {
     return _db!;
   }

   final path = p.join(
     await getDatabasesPath(),
     'tabungku.db',
   );

   _db = await openDatabase(
     path,
     version: 1,
     onCreate: (db, version) async {
       await db.execute('''
         CREATE TABLE kegiatan(
           id INTEGER PRIMARY KEY AUTOINCREMENT,
           judul TEXT NOT NULL,
           tanggal TEXT NOT NULL,
           selesai INTEGER NOT NULL,
           tipe TEXT NOT NULL,
           nominal INTEGER NOT NULL,
           kategori TEXT NOT NULL
         )
       ''');
     },
   );

   return _db!;
 }

 static Future<int> tambah(Kegiatan kegiatan) async {
   final db = await database;

   final id = await db.insert(
     'kegiatan',
     kegiatan.toMap(),
   );

   versiData.value++;
   return id;
 }
```

```dart
static Future<int> ubah(Kegiatan kegiatan) async {
  final db = await database;

  final hasil = await db.update(
    'kegiatan',
    kegiatan.toMap(),
    where: 'id = ?',
    whereArgs: [kegiatan.id],
  );

  versiData.value++;
  return hasil;
}

static Future<int> hapus(int id) async {
  final db = await database;

  final hasil = await db.delete(
    'kegiatan',
    where: 'id = ?',
    whereArgs: [id],
  );

  versiData.value++;
  return hasil;
}

static Future<int> ubahSelesai(
    int id,
    bool selesai,
    ) async {
  final db = await database;

  final hasil = await db.update(
    'kegiatan',
    {'selesai': selesai ? 1 : 0},
    where: 'id = ?',
    whereArgs: [id],
  );

  versiData.value++;
  return hasil;
}

static Future<List<Kegiatan>> pada(
    String tanggal,
    ) async {
  final db = await database;

  final rows = await db.query(
    'kegiatan',
    where: 'tanggal = ?',
```

```
    whereArgs: [tanggal],
    orderBy: 'id DESC',
  );

  return rows.map(Kegiatan.fromMap).toList();
}

static Future<Ringkasan> ringkasan(
    String tanggal,
    ) async {
  final db = await database;

  final hasil = await db.rawQuery('''
    SELECT
      COUNT(*) AS jumlah,
      COALESCE(SUM(selesai), 0) AS selesai,
      COALESCE(
        SUM(
          CASE
            WHEN tipe = 'pemasukan'
            THEN nominal
            ELSE 0
          END
        ),
        0
      ) AS masuk,
      COALESCE(
        SUM(
          CASE
            WHEN tipe = 'pengeluaran'
            THEN nominal
            ELSE 0
          END
        ),
        0
      ) AS keluar
    FROM kegiatan
    WHERE tanggal = ?
  ''', [tanggal]);

  final baris = hasil.first;

  return Ringkasan(
    jumlahKegiatan: (baris['jumlah'] as int?) ?? 0,
    selesai: (baris['selesai'] as int?) ?? 0,
    pemasukan: (baris['masuk'] as int?) ?? 0,
    pengeluaran: (baris['keluar'] as int?) ?? 0,
  );
}

static Future<Map<String, int>> pengeluaranPerKategori(
    String tanggal,
    ) async {
```

```
  final db = await database;

  final hasil = await db.rawQuery('''
    SELECT
      kategori,
      COALESCE(SUM(nominal), 0) AS total
    FROM kegiatan
    WHERE tanggal = ?
      AND tipe = 'pengeluaran'
    GROUP BY kategori
    ORDER BY total DESC
  ''', [tanggal]);

  return {
    for (final row in hasil)
      row['kategori'] as String:
      (row['total'] as int?) ?? 0,
  };
}

static Future<List<Map<String, Object?>>> tujuhHariTerakhir(
    String tanggal,
    ) async {
  final db = await database;

  return db.rawQuery('''
    SELECT
      tanggal,
      COUNT(*) AS jumlah,
      COALESCE(
        SUM(
          CASE
            WHEN tipe = 'pemasukan'
            THEN nominal
            ELSE 0
          END
        ),
        0
      ) AS pemasukan,
      COALESCE(
        SUM(
          CASE
            WHEN tipe = 'pengeluaran'
            THEN nominal
            ELSE 0
          END
        ),
        0
      ) AS pengeluaran
    FROM kegiatan
    WHERE tanggal BETWEEN date(?, '-6 day') AND ?
    GROUP BY tanggal
    ORDER BY tanggal DESC
```

```
    ''', [tanggal, tanggal]);
  }

  static Future<List<Kegiatan>> cari(
      String kata,
      ) async {
    final db = await database;

    final rows = await db.query(
      'kegiatan',
      where: '''
        judul LIKE ?
        OR tipe LIKE ?
        OR kategori LIKE ?
      ''',
      whereArgs: [
        '%$kata%',
        '%$kata%',
        '%$kata%',
      ],
      orderBy: 'tanggal DESC, id DESC',
    );

    return rows.map(Kegiatan.fromMap).toList();
  }
}
```

state/pengaturan.dart :

```
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

class Pengaturan extends ChangeNotifier {
  bool gelap = false;
  Color warna = Colors.blue;
  double target = 20000;

  Future<void> muat() async {
    final prefs = await SharedPreferences.getInstance();

    gelap = prefs.getBool('gelap') ?? false;

    final warnaValue = prefs.getInt('warna');
    if (warnaValue != null) {
      warna = Color(warnaValue);
    }

    target = prefs.getDouble('target') ?? 20000;

    notifyListeners();
  }

  Future<void> ubahGelap(bool nilai) async {
    gelap = nilai;
```

```
  final prefs = await SharedPreferences.getInstance();
  await prefs.setBool('gelap', nilai);

  notifyListeners();
 }

 Future<void> ubahWarna(Color nilai) async {
  warna = nilai;

  final prefs = await SharedPreferences.getInstance();
  await prefs.setInt('warna', nilai.toARGB32());

  notifyListeners();
 }

 Future<void> ubahTarget(double nilai) async {
  target = nilai;

  final prefs = await SharedPreferences.getInstance();
  await prefs.setDouble('target', nilai);

  notifyListeners();
 }
}
```

utils/format.dart :

```
String rupiah(int angka) {
 final teks = angka.abs().toString();
 final hasil = <String>[];

 for (int i = teks.length; i > 0; i -= 3) {
  final mulai = i - 3 < 0 ? 0 : i - 3;
  hasil.insert(0, teks.substring(mulai, i));
 }

 final nilai = hasil.join('.');

 return angka < 0 ? '-Rp $nilai' : 'Rp $nilai';
}

String tanggalHariIni() {
 final now = DateTime.now();

 return '${now.year.toString().padLeft(4, '0')}-'
     '${now.month.toString().padLeft(2, '0')}-'
     '${now.day.toString().padLeft(2, '0')}';
}

String tanggalTampilan(String tanggal) {
 final bagian = tanggal.split('-');

 if (bagian.length != 3) {
```

```
    return tanggal;
  }

  const bulan = [
    'Januari',
    'Februari',
    'Maret',
    'April',
    'Mei',
    'Juni',
    'Juli',
    'Agustus',
    'September',
    'Oktober',
    'November',
    'Desember',
  ];

  final nomorBulan = int.tryParse(bagian[1]);

  if (nomorBulan == null ||
      nomorBulan < 1 ||
      nomorBulan > 12) {
    return tanggal;
  }

  return '${int.parse(bagian[2])} '
      '${bulan[nomorBulan - 1]} '
      '${bagian[0]}';
}
```

widgets/ringkasan_card.dart :

```
import 'package:flutter/material.dart';
import '../models/kegiatan.dart';
import '../utils/format.dart';

class RingkasanCard extends StatelessWidget {
  final Ringkasan ringkasan;
  final double target;

  const RingkasanCard({
    super.key,
    required this.ringkasan,
    required this.target,
  });

  @override
  Widget build(BuildContext context) {
    final progress = target <= 0
        ? 0.0
        : (ringkasan.tabungan / target).clamp(0.0, 1.0);

    final tercapai =
```

```
      ringkasan.tabungan >= target && target > 0;

  return Card(
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Ringkasan Hari Ini',
            style: Theme.of(context).textTheme.titleLarge,
          ),
          const SizedBox(height: 16),
          Row(
            children: [
              Expanded(
                child: _Item(
                  label: 'Kegiatan',
                  value:
                  '${ringkasan.selesai}/${ringkasan.jumlahKegiatan}',
                ),
              ),
              Expanded(
                child: _Item(
                  label: 'Pemasukan',
                  value: rupiah(ringkasan.pemasukan),
                ),
              ),
              Expanded(
                child: _Item(
                  label: 'Pengeluaran',
                  value: rupiah(ringkasan.pengeluaran),
                ),
              ),
            ],
          ),
          const SizedBox(height: 16),
          Text(
            'Tabungan',
            style: Theme.of(context).textTheme.titleMedium,
          ),
          const SizedBox(height: 4),
          Text(
            rupiah(ringkasan.tabungan),
            style: TextStyle(
              fontSize: 24,
              fontWeight: FontWeight.bold,
              color: ringkasan.tabungan < 0
                  ? Colors.red
                  : Colors.green,
            ),
          ),
          const SizedBox(height: 12),
```

```
            TweenAnimationBuilder<double>(
              tween: Tween(
                begin: 0,
                end: progress,
              ),
              duration: const Duration(milliseconds: 700),
              builder: (context, value, child) {
                return LinearProgressIndicator(
                  value: value,
                  minHeight: 10,
                );
              },
            ),
            const SizedBox(height: 8),
            Text(
              tercapai
                  ? 'Target tercapai! Bagus.'
                  : 'Target ${rupiah(target.toInt())}',
              style: TextStyle(
                color: ringkasan.tabungan < 0
                    ? Colors.red
                    : null,
              ),
            ),
            if (ringkasan.tabungan < 0) ...[
              const SizedBox(height: 8),
              const Text(
                'Pengeluaran lebih besar daripada pemasukan.',
                style: TextStyle(
                  color: Colors.red,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ],
          ],
        ),
      ),
    );
  }
}

class _Item extends StatelessWidget {
  final String label;
  final String value;

  const _Item({
    required this.label,
    required this.value,
  });

  @override
  Widget build(BuildContext context) {
    return Column(
```

```
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(label),
        const SizedBox(height: 4),
        Text(
          value,
          style: const TextStyle(
            fontWeight: FontWeight.bold,
          ),
        ),
      ],
    );
  }
}
```

widgets/kegiatan_tile.dart :

```
import 'package:flutter/material.dart';
import '../models/kegiatan.dart';
import '../utils/format.dart';

class KegiatanTile extends StatelessWidget {
  final Kegiatan kegiatan;
  final VoidCallback onEdit;
  final VoidCallback onDelete;
  final ValueChanged<bool> onChanged;

  const KegiatanTile({
    super.key,
    required this.kegiatan,
    required this.onEdit,
    required this.onDelete,
    required this.onChanged,
  });

  @override
  Widget build(BuildContext context) {
    Color? warnaNominal;

    if (kegiatan.tipe == Tipe.pemasukan) {
      warnaNominal = Colors.green;
    } else if (kegiatan.tipe == Tipe.pengeluaran) {
      warnaNominal = Colors.red;
    }

    return Card(
      child: ListTile(
        leading: Checkbox(
          value: kegiatan.selesai,
          onChanged: (value) {
            onChanged(value ?? false);
          },
        ),
        title: Text(
```

```
        kegiatan.judul,
        style: TextStyle(
          decoration: kegiatan.selesai
              ? TextDecoration.lineThrough
              : null,
        ),
      ),
      subtitle: Text(
        '${kegiatan.tipe.name} • ${kegiatan.kategori}',
      ),
      trailing: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          if (kegiatan.nominal > 0)
            Text(
              '${kegiatan.tipe == Tipe.pemasukan ? '+' :
'-'}${rupiah(kegiatan.nominal)}',
              style: TextStyle(
                color: warnaNominal,
                fontWeight: FontWeight.bold,
              ),
            ),
          IconButton(
            icon: const Icon(Icons.edit),
            onPressed: onEdit,
          ),
          IconButton(
            icon: const Icon(Icons.delete),
            onPressed: onDelete,
          ),
        ],
      ),
    ),
  );
 }
}
```

pages/shell_page.dart :

```
import 'package:flutter/material.dart';
import '../state/pengaturan.dart';
import 'beranda_page.dart';
import 'laporan_page.dart';
import 'pengaturan_page.dart';

class ShellPage extends StatefulWidget {
 final Pengaturan pengaturan;

 const ShellPage({
   super.key,
   required this.pengaturan,
 });

 @override
```

```
 State<ShellPage> createState() => _ShellPageState();
}

class _ShellPageState extends State<ShellPage> {
 int index = 0;

 @override
 Widget build(BuildContext context) {
   final pages = [
     BerandaPage(
       pengaturan: widget.pengaturan,
     ),
     const LaporanPage(),
     PengaturanPage(
       pengaturan: widget.pengaturan,
     ),
   ];

   return Scaffold(
     body: IndexedStack(
       index: index,
       children: pages,
     ),
     bottomNavigationBar: NavigationBar(
       selectedIndex: index,
       onDestinationSelected: (value) {
         setState(() {
           index = value;
         });
       },
       destinations: const [
         NavigationDestination(
           icon: Icon(Icons.today_outlined),
           selectedIcon: Icon(Icons.today),
           label: 'Hari Ini',
         ),
         NavigationDestination(
           icon: Icon(Icons.bar_chart_outlined),
           selectedIcon: Icon(Icons.bar_chart),
           label: 'Laporan',
         ),
         NavigationDestination(
           icon: Icon(Icons.settings_outlined),
           selectedIcon: Icon(Icons.settings),
           label: 'Pengaturan',
         ),
       ],
     ),
   );
 }
}
```

pages/beranda_page.dart :

```dart
import 'package:flutter/material.dart';
import '../data/db_helper.dart';
import '../models/kegiatan.dart';
import '../state/pengaturan.dart';
import '../utils/format.dart';
import '../widgets/kegiatan_tile.dart';
import '../widgets/ringkasan_card.dart';

class BerandaPage extends StatefulWidget {
 final Pengaturan pengaturan;

 const BerandaPage({
   super.key,
   required this.pengaturan,
 });

 @override
 State<BerandaPage> createState() => _BerandaPageState();
}

class _BerandaPageState extends State<BerandaPage> {
 late Future<List<Kegiatan>> _futureKegiatan;
 late Future<Ringkasan> _futureRingkasan;

 @override
 void initState() {
   super.initState();
   _muat();
   versiData.addListener(_refresh);
 }

 @override
 void dispose() {
   versiData.removeListener(_refresh);
   super.dispose();
 }

 void _muat() {
   final tanggal = tanggalHariIni();

   _futureKegiatan = DbHelper.pada(tanggal);
   _futureRingkasan = DbHelper.ringkasan(tanggal);
 }

 void _refresh() {
   if (!mounted) {
     return;
   }

   setState(() {
     _muat();
   });
```

```dart
}

Future<void> _hapus(Kegiatan kegiatan) async {
  final konfirmasi = await showDialog<bool>(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: const Text('Hapus kegiatan?'),
        content: Text(
          'Kegiatan "${kegiatan.judul}" akan dihapus.',
        ),
        actions: [
          TextButton(
            onPressed: () {
              Navigator.pop(context, false);
            },
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () {
              Navigator.pop(context, true);
            },
            child: const Text('Hapus'),
          ),
        ],
      );
    },
  );

  if (konfirmasi == true && kegiatan.id != null) {
    await DbHelper.hapus(kegiatan.id!);
  }
}

void _tambah() {
  Navigator.pushNamed(
    context,
    '/form',
  );
}

void _ubah(Kegiatan kegiatan) {
  Navigator.pushNamed(
    context,
    '/form',
    arguments: kegiatan,
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
```

```
      title: const Text('TabungKu'),
    ),
    body: FutureBuilder(
      future: Future.wait([
        _futureKegiatan,
        _futureRingkasan,
      ]),
      builder: (context, snapshot) {
        if (snapshot.connectionState ==
            ConnectionState.waiting) {
          return const Center(
            child: CircularProgressIndicator(),
          );
        }

        if (snapshot.hasError) {
          return Center(
            child: Text(
              'Terjadi kesalahan: ${snapshot.error}',
            ),
          );
        }

        final data = snapshot.data!;

        final kegiatan =
        data[0] as List<Kegiatan>;
        final ringkasan =
        data[1] as Ringkasan;

        return RefreshIndicator(
          onRefresh: () async {
            setState(_muat);
          },
          child: ListView(
            padding: const EdgeInsets.all(16),
            children: [
              Text(
                tanggalTampilan(tanggalHariIni()),
                style: Theme.of(context)
                    .textTheme
                    .titleMedium,
              ),
              const SizedBox(height: 12),
              RingkasanCard(
                ringkasan: ringkasan,
                target: widget.pengaturan.target,
              ),
              const SizedBox(height: 20),
              if (kegiatan.isEmpty)
                Card(
                  child: Padding(
                    padding: const EdgeInsets.all(24),
```

```
                    child: Column(
                      children: [
                        const Icon(
                          Icons.inbox_outlined,
                          size: 48,
                        ),
                        const SizedBox(height: 12),
                        const Text(
                          'Belum ada kegiatan hari ini.',
                        ),
                        const SizedBox(height: 12),
                        FilledButton.icon(
                          onPressed: _tambah,
                          icon: const Icon(Icons.add),
                          label: const Text(
                            'Tambah Kegiatan',
                          ),
                        ),
                      ],
                    ),
                  ),
                )
              else
                ...kegiatan.map(
                      (item) => KegiatanTile(
                    kegiatan: item,
                    onEdit: () => _ubah(item),
                    onDelete: () => _hapus(item),
                    onChanged: (value) {
                      if (item.id != null) {
                        DbHelper.ubahSelesai(
                          item.id!,
                          value,
                        );
                      }
                    },
                  ),
                ),
            ],
          ),
        );
      },
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: _tambah,
      child: const Icon(Icons.add),
    ),
  );
 }
}
```

pages/form_kegiatan_page.dart :

```dart
import 'package:flutter/material.dart';
import '../data/db_helper.dart';
import '../models/kegiatan.dart';
import '../state/pengaturan.dart';
import '../utils/format.dart';

class FormKegiatanPage extends StatefulWidget {
 final Pengaturan pengaturan;

 const FormKegiatanPage({
   super.key,
   required this.pengaturan,
 });

 @override
 State<FormKegiatanPage> createState() =>
     _FormKegiatanPageState();
}

class _FormKegiatanPageState
   extends State<FormKegiatanPage> {
 final _formKey = GlobalKey<FormState>();
 final _judulController = TextEditingController();
 final _nominalController = TextEditingController();

 DateTime _tanggal = DateTime.now();
 Tipe _tipe = Tipe.tanpa;
 String _kategori = 'Makan';
 bool _selesai = false;
 Kegiatan? _edit;

 @override
 void didChangeDependencies() {
   super.didChangeDependencies();

   if (_edit != null) {
     return;
   }

   final argument = ModalRoute.of(context)?.settings.arguments;

   if (argument is Kegiatan) {
     _edit = argument;
     _judulController.text = argument.judul;
     _nominalController.text =
     argument.nominal == 0
         ? ''
         : argument.nominal.toString();

     _tanggal = DateTime.tryParse(argument.tanggal) ??
         DateTime.now();
     _tipe = argument.tipe;
```

```
    _kategori = argument.kategori == '-'
        ? 'Makan'
        : argument.kategori;
    _selesai = argument.selesai;
  }
}

@override
void dispose() {
  _judulController.dispose();
  _nominalController.dispose();
  super.dispose();
}

String _tanggalDatabase() {
  return '${_tanggal.year.toString().padLeft(4, '0')}-'
      '${_tanggal.month.toString().padLeft(2, '0')}-'
      '${_tanggal.day.toString().padLeft(2, '0')}';
}

Future<void> _pilihTanggal() async {
  final hasil = await showDatePicker(
    context: context,
    initialDate: _tanggal,
    firstDate: DateTime(2020),
    lastDate: DateTime(2100),
  );

  if (hasil != null) {
    setState(() {
      _tanggal = hasil;
    });
  }
}

Future<void> _simpan() async {
  if (!_formKey.currentState!.validate()) {
    return;
  }

  final nominal =
      int.tryParse(_nominalController.text.trim()) ?? 0;

  final kegiatan = Kegiatan(
    id: _edit?.id,
    judul: _judulController.text.trim(),
    tanggal: _tanggalDatabase(),
    selesai: _selesai,
    tipe: _tipe,
    nominal: _tipe == Tipe.tanpa ? 0 : nominal,
    kategori:
    _tipe == Tipe.pengeluaran ? _kategori : '-',
  );
```

```
  if (_edit == null) {
    await DbHelper.tambah(kegiatan);
  } else {
    await DbHelper.ubah(kegiatan);
  }

  if (!mounted) {
    return;
  }

  Navigator.pop(context);
}

@override
Widget build(BuildContext context) {
  final membutuhkanNominal = _tipe != Tipe.tanpa;
  final membutuhkanKategori = _tipe == Tipe.pengeluaran;

  return Scaffold(
    appBar: AppBar(
      title: Text(
        _edit == null
            ? 'Tambah Kegiatan'
            : 'Ubah Kegiatan',
      ),
    ),
    body: Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          TextFormField(
            controller: _judulController,
            decoration: const InputDecoration(
              labelText: 'Nama Kegiatan',
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null ||
                  value.trim().isEmpty) {
                return 'Nama kegiatan wajib diisi';
              }

              return null;
            },
          ),
          const SizedBox(height: 16),
          ListTile(
            contentPadding: EdgeInsets.zero,
            title: const Text('Tanggal'),
            subtitle: Text(
              tanggalTampilan(_tanggalDatabase()),
```

```
            ),
            trailing: OutlinedButton(
              onPressed: _pilihTanggal,
              child: const Text('Pilih'),
            ),
          ),
          const SizedBox(height: 8),
          DropdownButtonFormField<Tipe>(
            initialValue: _tipe,
            decoration: const InputDecoration(
              labelText: 'Jenis Keuangan',
              border: OutlineInputBorder(),
            ),
            items: const [
              DropdownMenuItem(
                value: Tipe.tanpa,
                child: Text('Tidak melibatkan uang'),
              ),
              DropdownMenuItem(
                value: Tipe.pemasukan,
                child: Text('Pemasukan'),
              ),
              DropdownMenuItem(
                value: Tipe.pengeluaran,
                child: Text('Pengeluaran'),
              ),
            ],
            onChanged: (value) {
              if (value != null) {
                setState(() {
                  _tipe = value;
                });
              }
            },
          ),
          const SizedBox(height: 16),
          if (membutuhkanNominal)
            TextFormField(
              controller: _nominalController,
              keyboardType: TextInputType.number,
              decoration: const InputDecoration(
                labelText: 'Nominal',
                prefixText: 'Rp ',
                border: OutlineInputBorder(),
              ),
              validator: (value) {
                if (!membutuhkanNominal) {
                  return null;
                }

                final nominal =
                int.tryParse(value ?? '');
```

```
            if (nominal == null || nominal <= 0) {
              return 'Nominal harus lebih dari 0';
            }

            return null;
          },
        ),
      if (membutuhkanNominal)
        const SizedBox(height: 16),
      if (membutuhkanKategori)
        DropdownButtonFormField<String>(
          initialValue: _kategori,
          decoration: const InputDecoration(
            labelText: 'Kategori',
            border: OutlineInputBorder(),
          ),
          items: const [
            DropdownMenuItem(
              value: 'Makan',
              child: Text('Makan'),
            ),
            DropdownMenuItem(
              value: 'Transport',
              child: Text('Transport'),
            ),
            DropdownMenuItem(
              value: 'Belajar',
              child: Text('Belajar'),
            ),
            DropdownMenuItem(
              value: 'Hiburan',
              child: Text('Hiburan'),
            ),
            DropdownMenuItem(
              value: 'Lainnya',
              child: Text('Lainnya'),
            ),
          ],
          onChanged: (value) {
            if (value != null) {
              setState(() {
                _kategori = value;
              });
            }
          },
        ),
      if (membutuhkanKategori)
        const SizedBox(height: 16),
      SwitchListTile(
        contentPadding: EdgeInsets.zero,
        title: const Text('Sudah selesai'),
        value: _selesai,
        onChanged: (value) {
```

```
                setState(() {
                  _selesai = value;
                });
              },
            ),
            const SizedBox(height: 16),
            FilledButton.icon(
              onPressed: _simpan,
              icon: const Icon(Icons.save),
              label: const Text('Simpan'),
            ),
          ],
        ),
      ),
    );
  }
}
```

pages/laporan_page.dart :

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import '../data/db_helper.dart';
import '../models/kegiatan.dart';
import '../utils/format.dart';

class LaporanPage extends StatefulWidget {
  const LaporanPage({super.key});

  @override
  State<LaporanPage> createState() => _LaporanPageState();
}

class _LaporanPageState extends State<LaporanPage> {
  DateTime _tanggal = DateTime.now();

  late Future<Ringkasan> _futureRingkasan;
  late Future<Map<String, int>> _futureKategori;
  late Future<List<Map<String, Object?>>>
  _futureTujuhHari;

  @override
  void initState() {
    super.initState();
    _muat();
    versiData.addListener(_refresh);
  }

  @override
  void dispose() {
    versiData.removeListener(_refresh);
    super.dispose();
  }
```

```dart
String _tanggalDatabase() {
  return '${_tanggal.year.toString().padLeft(4, '0')}-'
      '${_tanggal.month.toString().padLeft(2, '0')}-'
      '${_tanggal.day.toString().padLeft(2, '0')}';
}

void _muat() {
  final tanggal = _tanggalDatabase();

  _futureRingkasan = DbHelper.ringkasan(tanggal);
  _futureKategori =
      DbHelper.pengeluaranPerKategori(tanggal);
  _futureTujuhHari =
      DbHelper.tujuhHariTerakhir(tanggal);
}

void _refresh() {
  if (!mounted) {
    return;
  }

  setState(_muat);
}

Future<void> _pilihTanggal() async {
  final hasil = await showDatePicker(
    context: context,
    initialDate: _tanggal,
    firstDate: DateTime(2020),
    lastDate: DateTime(2100),
  );

  if (hasil != null) {
    setState(() {
      _tanggal = hasil;
      _muat();
    });
  }
}

Future<void> _export() async {
  final ringkasan = await _futureRingkasan;
  final kategori = await _futureKategori;

  final buffer = StringBuffer();

  buffer.writeln(
    'Laporan TabungKu - ${tanggalTampilan(_tanggalDatabase())}',
  );
  buffer.writeln('');
  buffer.writeln(
    'Jumlah kegiatan: ${ringkasan.jumlahKegiatan}',
  );
```

```
  buffer.writeln(
    'Selesai: ${ringkasan.selesai}',
  );
  buffer.writeln(
    'Pemasukan: ${rupiah(ringkasan.pemasukan)}',
  );
  buffer.writeln(
    'Pengeluaran: ${rupiah(ringkasan.pengeluaran)}',
  );
  buffer.writeln(
    'Tabungan: ${rupiah(ringkasan.tabungan)}',
  );

  if (kategori.isNotEmpty) {
    buffer.writeln('');
    buffer.writeln('Pengeluaran per kategori:');

    for (final item in kategori.entries) {
      buffer.writeln(
        '${item.key}: ${rupiah(item.value)}',
      );
    }
  }

  await Clipboard.setData(
    ClipboardData(
      text: buffer.toString(),
    ),
  );

  if (!mounted) {
    return;
  }

  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Laporan berhasil disalin'),
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Laporan'),
      actions: [
        IconButton(
          onPressed: _export,
          icon: const Icon(Icons.copy),
          tooltip: 'Salin laporan',
        ),
      ],
```

```
    ),
    body: ListView(
      padding: const EdgeInsets.all(16),
      children: [
        OutlinedButton.icon(
          onPressed: _pilihTanggal,
          icon: const Icon(Icons.calendar_month),
          label: Text(
            tanggalTampilan(_tanggalDatabase()),
          ),
        ),
        const SizedBox(height: 16),
        FutureBuilder<Ringkasan>(
          future: _futureRingkasan,
          builder: (context, snapshot) {
            if (snapshot.connectionState ==
                ConnectionState.waiting) {
              return const Center(
                child: CircularProgressIndicator(),
              );
            }

            if (snapshot.hasError) {
              return Text(
                'Error: ${snapshot.error}',
              );
            }

            final data = snapshot.data!;

            return Card(
              child: Padding(
                padding: const EdgeInsets.all(16),
                child: Column(
                  crossAxisAlignment:
                  CrossAxisAlignment.start,
                  children: [
                    Text(
                      'Ringkasan',
                      style: Theme.of(context)
                          .textTheme
                          .titleLarge,
                    ),
                    const SizedBox(height: 16),
                    Text(
                      'Kegiatan: ${data.selesai}/${data.jumlahKegiatan}',
                    ),
                    Text(
                      'Pemasukan: ${rupiah(data.pemasukan)}',
                    ),
                    Text(
                      'Pengeluaran: ${rupiah(data.pengeluaran)}',
                    ),
```

```
                Text(
                  'Tabungan: ${rupiah(data.tabungan)}',
                  style: TextStyle(
                    fontWeight: FontWeight.bold,
                    color: data.tabungan < 0
                        ? Colors.red
                        : Colors.green,
                  ),
                ),
              ],
            ),
          ),
        );
      },
    ),
    const SizedBox(height: 16),
    FutureBuilder<Map<String, int>>(
      future: _futureKategori,
      builder: (context, snapshot) {
        if (snapshot.connectionState ==
            ConnectionState.waiting) {
          return const Center(
            child: CircularProgressIndicator(),
          );
        }

        final kategori = snapshot.data ?? {};

        return Card(
          child: Padding(
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment:
              CrossAxisAlignment.start,
              children: [
                Text(
                  'Pengeluaran per Kategori',
                  style: Theme.of(context)
                      .textTheme
                      .titleLarge,
                ),
                const SizedBox(height: 12),
                if (kategori.isEmpty)
                  const Text(
                    'Belum ada pengeluaran.',
                  )
                else
                  ...kategori.entries.map(
                        (item) => ListTile(
                      contentPadding: EdgeInsets.zero,
                      title: Text(item.key),
                      trailing: Text(
                        rupiah(item.value),
```

```
                        style: const TextStyle(
                          fontWeight: FontWeight.bold,
                        ),
                      ),
                    ),
                  ),
                ),
              ],
            ),
          ),
        );
      },
    ),
    const SizedBox(height: 16),
    FutureBuilder<
        List<Map<String, Object?>>>(
      future: _futureTujuhHari,
      builder: (context, snapshot) {
        if (snapshot.connectionState ==
            ConnectionState.waiting) {
          return const Center(
            child: CircularProgressIndicator(),
          );
        }

        final data = snapshot.data ?? [];

        return Card(
          child: Padding(
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment:
              CrossAxisAlignment.start,
              children: [
                Text(
                  '7 Hari Terakhir',
                  style: Theme.of(context)
                      .textTheme
                      .titleLarge,
                ),
                const SizedBox(height: 12),
                if (data.isEmpty)
                  const Text(
                    'Belum ada data.',
                  )
                else
                  ...data.map(
                        (item) {
                      final tanggal =
                      item['tanggal'] as String;
                      final pemasukan =
                          (item['pemasukan'] as int?) ??
                              0;
                      final pengeluaran =
```

```
                                (item['pengeluaran'] as int?) ??
                                    0;

                            return ListTile(
                              contentPadding:
                              EdgeInsets.zero,
                              title: Text(
                                tanggalTampilan(tanggal),
                              ),
                              subtitle: Text(
                                'Pemasukan ${rupiah(pemasukan)} • '
                                    'Pengeluaran ${rupiah(pengeluaran)}',
                              ),
                              trailing: Text(
                                rupiah(
                                  pemasukan -
                                      pengeluaran,
                                ),
                              ),
                            );
                          },
                        ),
                      ],
                    ),
                  ),
                );
              },
            ),
          ],
        ),
      );
  }
}
```

pages/pengaturan_page.dart :

```
import 'package:flutter/material.dart';
import '../state/pengaturan.dart';
import '../utils/format.dart';

class PengaturanPage extends StatelessWidget {
 final Pengaturan pengaturan;

 const PengaturanPage({
   super.key,
   required this.pengaturan,
 });

 @override
 Widget build(BuildContext context) {
   return ListView(
     padding: const EdgeInsets.all(16),
     children: [
       Text(
```

```
        'Pengaturan',
        style: Theme.of(context)
            .textTheme
            .headlineSmall,
      ),
      const SizedBox(height: 20),
      Card(
        child: Column(
          children: [
            SwitchListTile(
              title: const Text('Mode Gelap'),
              subtitle: const Text(
                'Simpan pilihan tema setelah aplikasi ditutup',
              ),
              value: pengaturan.gelap,
              onChanged: pengaturan.ubahGelap,
            ),
            const Divider(height: 1),
            ListTile(
              title: const Text('Warna Tema'),
              subtitle: Text(
                'Warna saat ini',
              ),
              trailing: Wrap(
                spacing: 8,
                children: [
                  _Warna(
                    color: Colors.blue,
                    selected:
                    pengaturan.warna.toARGB32() ==
                        Colors.blue.toARGB32(),
                    onTap: () {
                      pengaturan.ubahWarna(
                        Colors.blue,
                      );
                    },
                  ),
                  _Warna(
                    color: Colors.green,
                    selected:
                    pengaturan.warna.toARGB32() ==
                        Colors.green.toARGB32(),
                    onTap: () {
                      pengaturan.ubahWarna(
                        Colors.green,
                      );
                    },
                  ),
                  _Warna(
                    color: Colors.deepPurple,
                    selected:
                    pengaturan.warna.toARGB32() ==
                        Colors.deepPurple.toARGB32(),
```

```
                  onTap: () {
                    pengaturan.ubahWarna(
                      Colors.deepPurple,
                    );
                  },
                ),
                _Warna(
                  color: Colors.orange,
                  selected:
                  pengaturan.warna.toARGB32() ==
                      Colors.orange.toARGB32(),
                  onTap: () {
                    pengaturan.ubahWarna(
                      Colors.orange,
                    );
                  },
                ),
              ],
            ),
          ),
        ],
      ),
    ),
    const SizedBox(height: 16),
    Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment:
          CrossAxisAlignment.start,
          children: [
            Text(
              'Target Tabungan Harian',
              style: Theme.of(context)
                  .textTheme
                  .titleMedium,
            ),
            const SizedBox(height: 8),
            Text(
              rupiah(
                pengaturan.target.toInt(),
              ),
              style: Theme.of(context)
                  .textTheme
                  .headlineSmall,
            ),
            Slider(
              min: 0,
              max: 500000,
              divisions: 100,
              value: pengaturan.target
                  .clamp(0, 500000),
              label: rupiah(
```

```
                  pengaturan.target.toInt(),
                ),
                onChanged: (value) {
                  pengaturan.ubahTarget(value);
                },
              ),
            ],
          ),
        ),
      ),
    ],
  );
 }
}

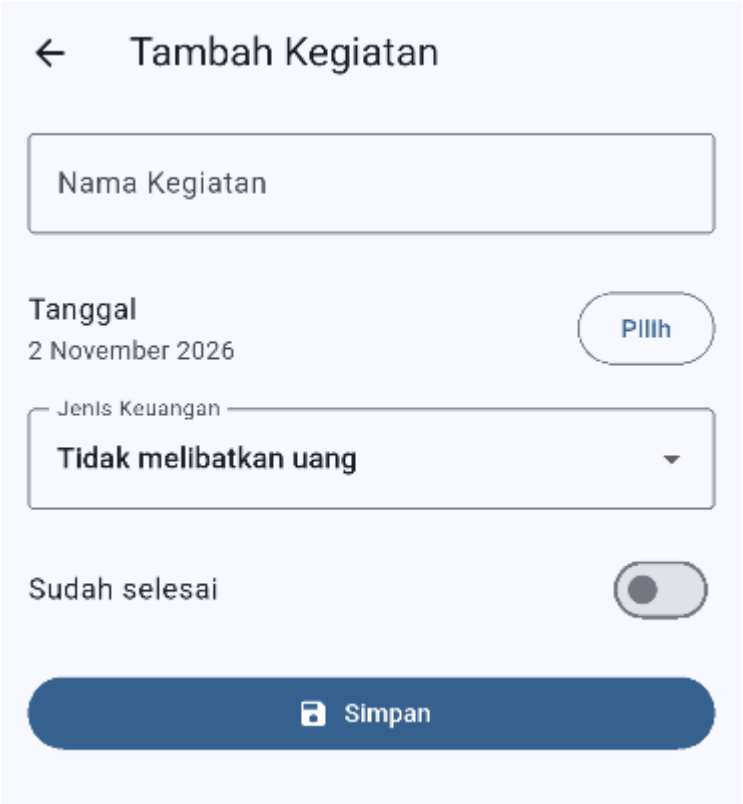
class _Warna extends StatelessWidget {
 final Color color;
 final bool selected;
 final VoidCallback onTap;

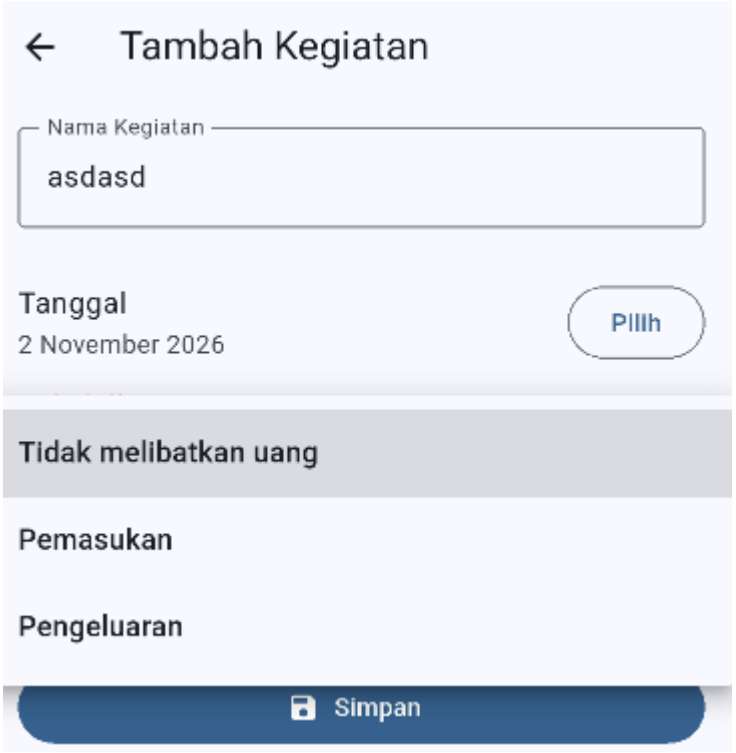
 const _Warna({
   required this.color,
   required this.selected,
   required this.onTap,
 });

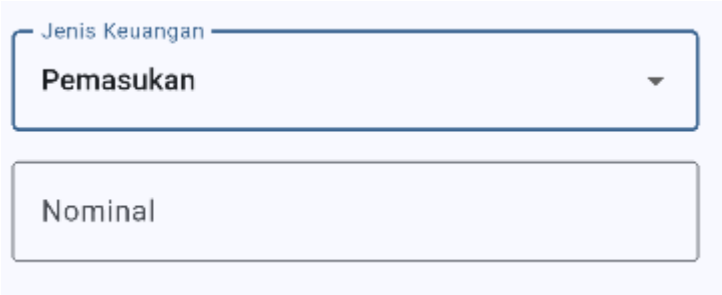
 @override
 Widget build(BuildContext context) {
   return GestureDetector(
     onTap: onTap,
     child: CircleAvatar(
       backgroundColor: color,
       radius: selected ? 18 : 15,
       child: selected
           ? const Icon(
         Icons.check,
         color: Colors.white,
         size: 18,
       )
           : null,
     ),
   );
 }
}
```

Hasil :


TabungKu
2 November 2026
Ringkasan Hari Ini
Kegiatan
0/0
Pemasukan
Rp 0
Pengeluaran
Rp 0
Tabungan
Rp 0
Target Rp 20.000
Belum ada kegiatan hari ini.
+ Tambah Kegiatan
Hari Ini
Laporan
Pengaturan


Tambah Kegiatan
Nama Kegiatan
Tanggal
2 November 2026
Pilih
Jenis Keuangan
Tidak melibatkan uang
Sudah selesai
Simpan


Tambah Kegiatan
Nama Kegiatan
asdasd
Tanggal
2 November 2026
Pilih
Tidak melibatkan uang
Pemasukan
Pengeluaran
Simpan





TabungKu

2 November 2026

Ringkasan Hari Ini

Kegiatan	Pemasukan	Pengeluaran
1/2	Rp 500.000	Rp 0

Tabungan

Rp 500.000

Target tercapai! Bagus.

~~kerja~~
pemasukan • +Rp 500.000
-

asdasd
tanpa • -


Tanggal
2 November 2026
Pilih
Jenis Keuangan
Pengeluaran
Nominal
Rp 20000
Makan
Transport
Belajar
Hiburan
Lainnya

TabungKu

2 November 2026

Ringkasan Hari Ini

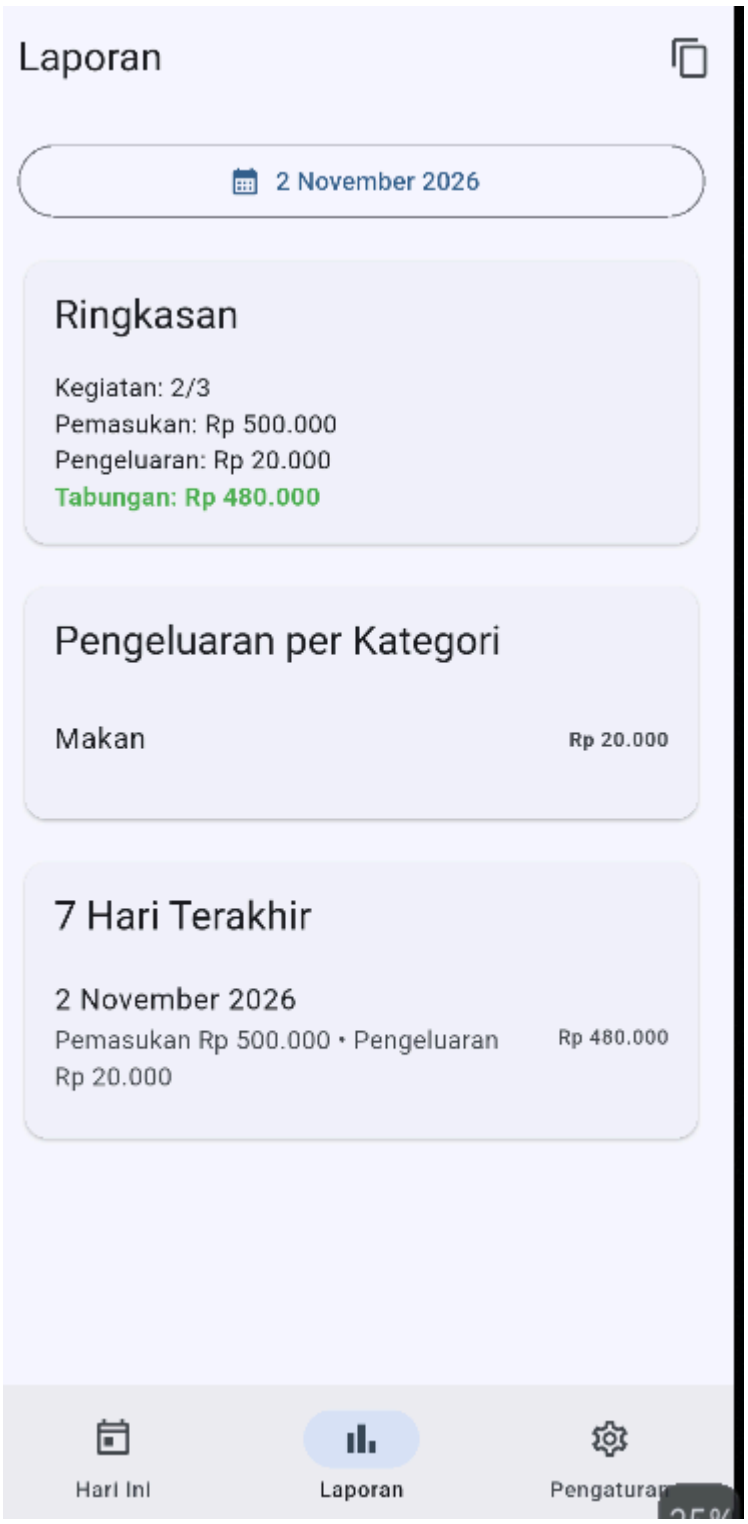
Kegiatan	Pemasukan	Pengeluaran
2/3	**Rp 500.000**	**Rp 20.000**

Tabungan

Rp 480.000

Target tercapai! Bagus.

- ~~jajan~~ pengeluaran • Makan -Rp 20.000
- kerja pemasukan • - +Rp 500.000
- asdasd tanpa • -


Laporan
2 November 2026
Ringkasan
Kegiatan: 2/3
Pemasukan: Rp 500.000
Pengeluaran: Rp 20.000
Tabungan: Rp 480.000
Pengeluaran per Kategori
Makan
Rp 20.000
7 Hari Terakhir
2 November 2026
Pemasukan Rp 500.000 • Pengeluaran Rp 20.000
Rp 480.000
Hari Ini
Laporan
Pengaturan

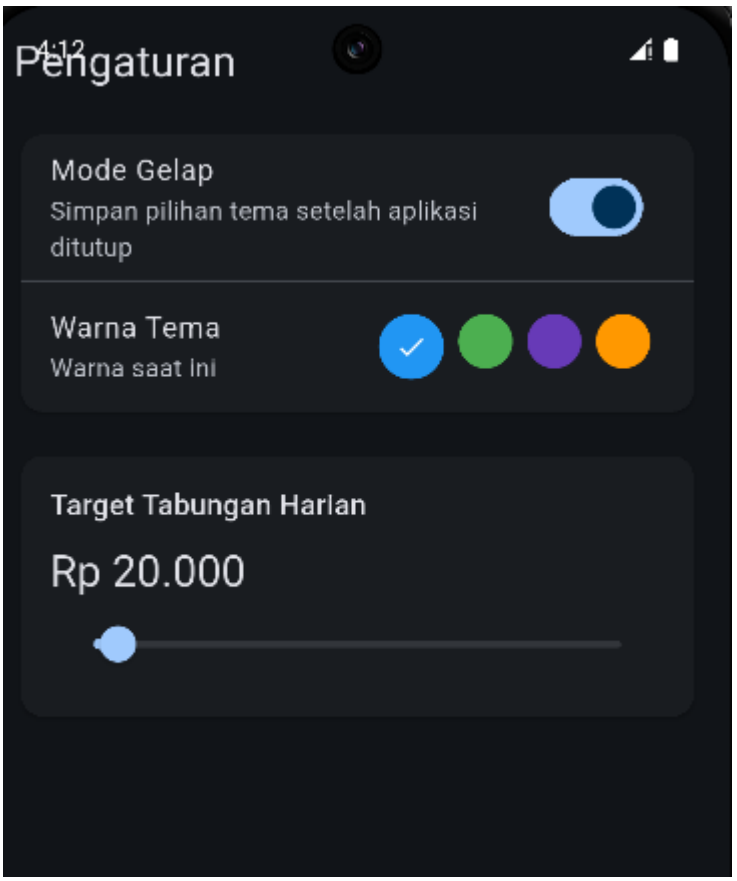
Hasil Copy :

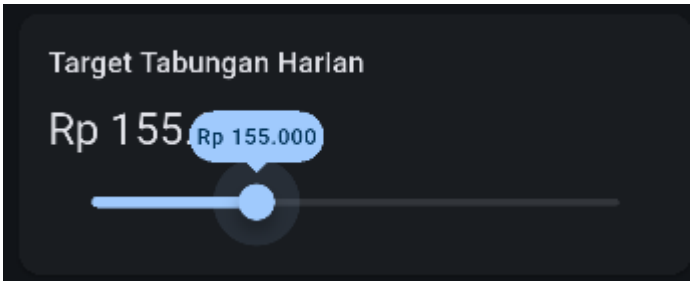
```
Laporan TabungKu - 2 November 2026

Jumlah kegiatan: 3
Selesai: 2
Pemasukan: Rp 500.000
Pengeluaran: Rp 20.000
Tabungan: Rp 480.000

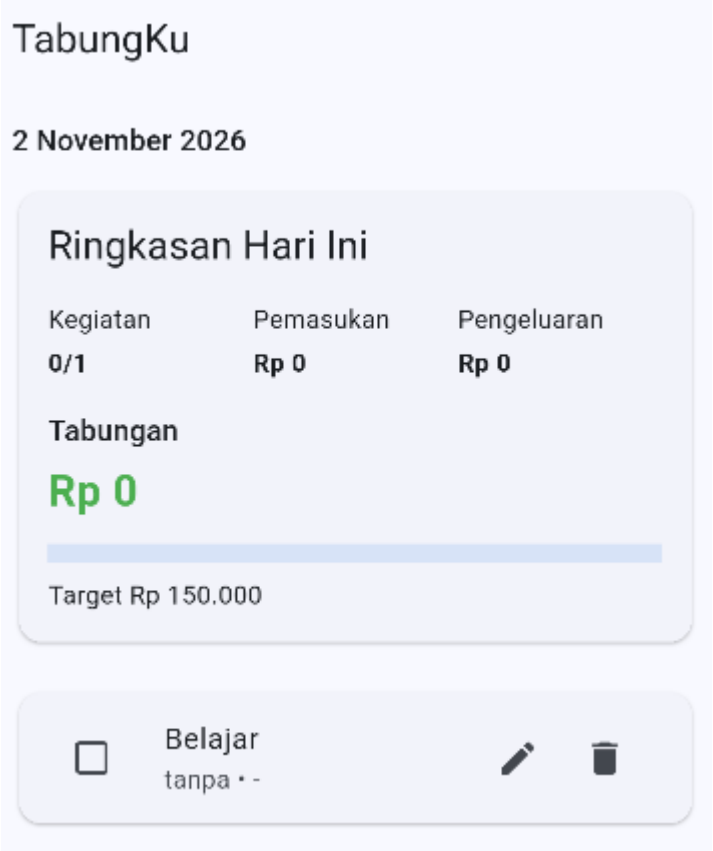
Pengeluaran per kategori:
Makan: Rp 20.000
```


Pengaturan
Mode Gelap
Simpan pilihan tema setelah aplikasi ditutup
Warna Tema
Warna saat ini
Target Tabungan Harian
Rp 20.000
Hari Ini
Laporan
Pengaturan


4:12
Pengaturan
Mode Gelap
Simpan pilihan tema setelah aplikasi ditutup
Warna Tema
Warna saat ini
Target Tabungan Harian
Rp 20.000


Target Tabungan Harian
Rp 155.
Rp 155.000

Skenario Uji :



TabungKu

2 November 2026

Ringkasan Hari Ini

Kegiatan	Pemasukan	Pengeluaran
1/2	**Rp 50.000**	**Rp 0**

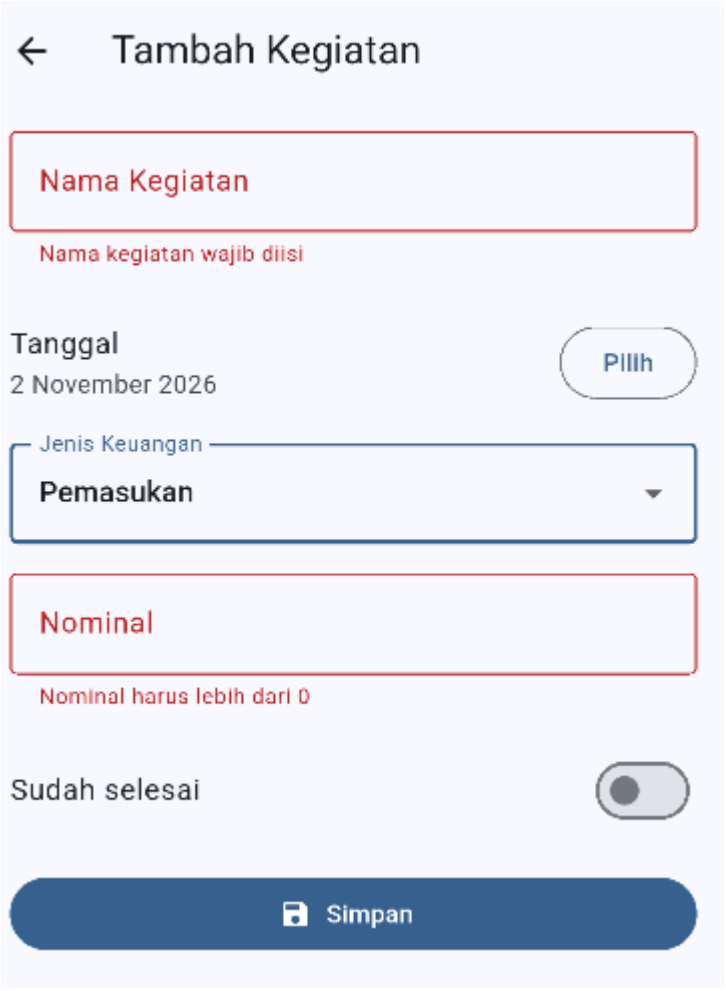
Tabungan

Rp 50.000

Target Rp 150.000

- [x] ~~pemasukan~~ — pemasukan • - — +Rp 50.000
- [ ] Belajar — tanpa • -


TabungKu
2 November 2026
Ringkasan Hari Ini
Kegiatan
2/3
Pemasukan
Rp 50.000
Pengeluaran
Rp 15.000
Tabungan
Rp 35.000
Target Rp 150.000
pegeluaran
pengeluaran • Makan
-Rp 15.000
pemasukan
pemasukan • -
+Rp 50.000
Belajar
tanpa • -


Tambah Kegiatan
Nama Kegiatan
Nama kegiatan wajib diisi
Tanggal
2 November 2026
Pilih
Jenis Keuangan
Pemasukan
Nominal
Nominal harus lebih dari 0
Sudah selesai
Simpan


Nominal
Rp 0
Nominal harus lebih dari 0

TabungKu

2 November 2026

Ringkasan Hari Ini

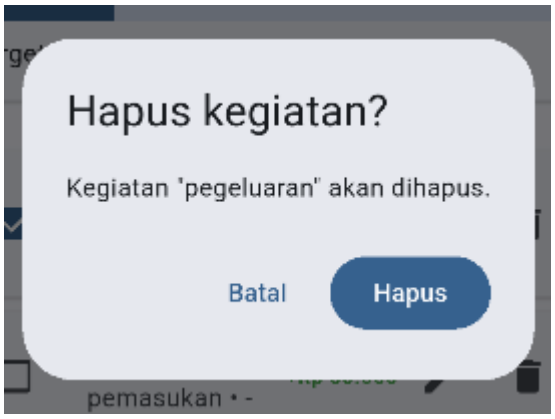
Kegiatan	Pemasukan	Pengeluaran
2/3	**Rp 50.000**	**Rp 15.000**

Tabungan

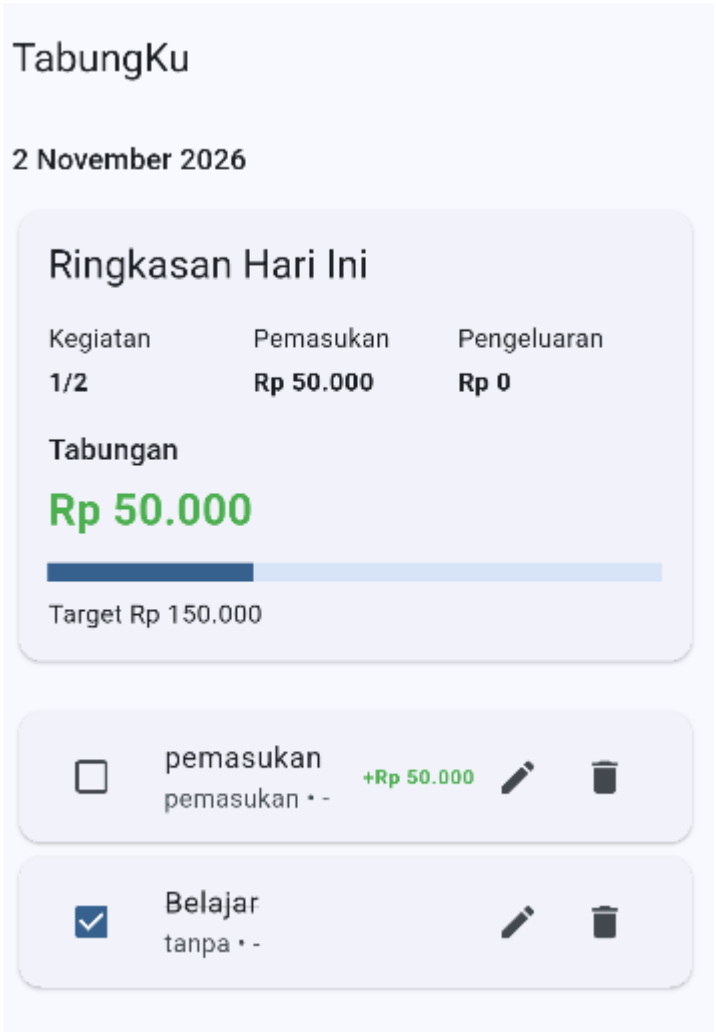
Rp 35.000

Target Rp 150.000

- [x] ~~pegeluaran~~ — pengeluaran • Makan — -Rp 15.000
- [ ] pemasukan — pemasukan • - — +Rp 50.000
- [x] ~~Belajar~~ — tanpa • -




Tambah Kegiatan
Nama Kegiatan
asd
Tanggal
1 November 2026
Pilih
Jenis Keuangan
Tidak melibatkan uang
Sudah selesai
Simpan


TabungKu
2 November 2026
Ringkasan Hari Ini
Kegiatan
1/2
Pemasukan
Rp 50.000
Pengeluaran
Rp 0
Tabungan
Rp 50.000
Target Rp 150.000
pemasukan
pemasukan • -
+Rp 50.000
Belajar
tanpa • -

Laporan

1 November 2026

Ringkasan

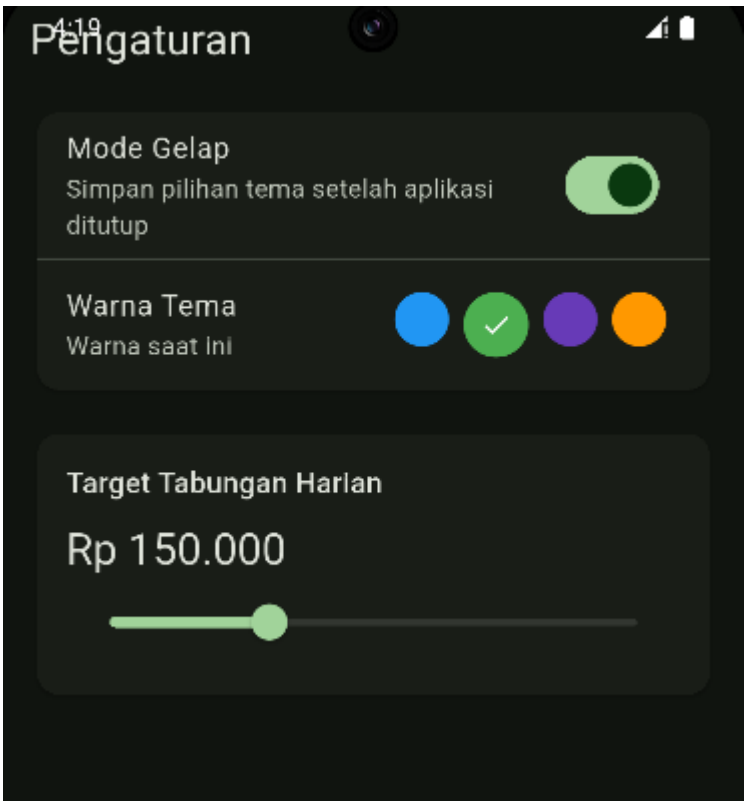
Kegiatan: 0/1
Pemasukan: Rp 0
Pengeluaran: Rp 0
Tabungan: Rp 0

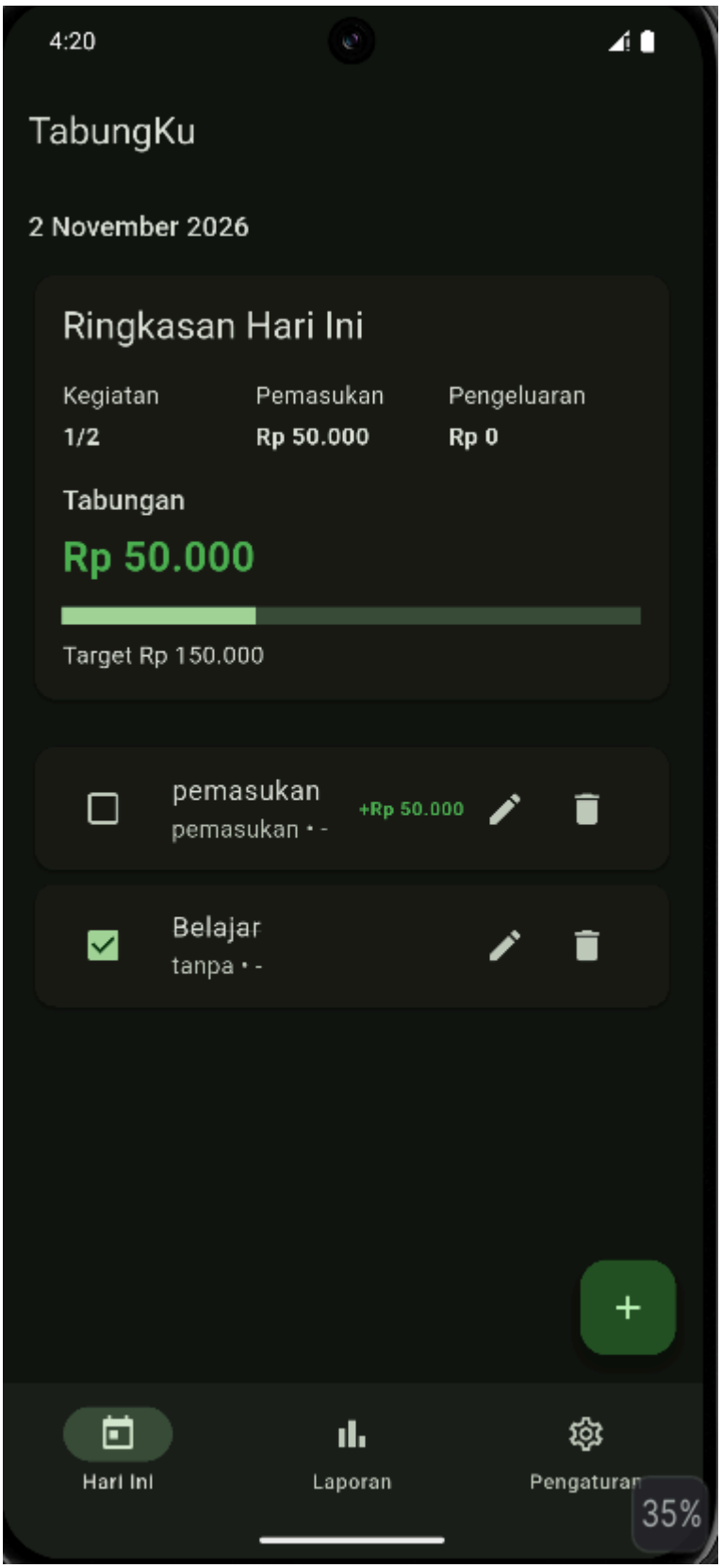
Pengeluaran per Kategori

Belum ada pengeluaran.

7 Hari Terakhir

1 November 2026
Pemasukan Rp 0 • Pengeluaran Rp 0
Rp 0


4:19
Pengaturan
Mode Gelap
Simpan pilihan tema setelah aplikasi ditutup
Warna Tema
Warna saat ini
Target Tabungan Harian
Rp 150.000


4:20
TabungKu
2 November 2026
Ringkasan Hari Ini
Kegiatan
1/2
Pemasukan
Rp 50.000
Pengeluaran
Rp 0
Tabungan
Rp 50.000
Target Rp 150.000
pemasukan
pemasukan • -
+Rp 50.000
Belajar
tanpa • -
Hari Ini
Laporan
Pengaturan
35%

TabungKu

2 November 2026

Ringkasan Hari Ini

Kegiatan	Pemasukan	Pengeluaran
1/3	Rp 50.000	Rp 500.000

Tabungan

-Rp 450.000

Target Rp 150.000

Pengeluaran lebih besar daripada pemasukan.

- [ ] jajan
  pengeluaran • Hiburan -Rp 500.000
- [ ] pemasukan
  pemasukan • - +Rp 50.000
- [x] ~~Belajar~~
  tanpa • -